SlangGPT

Fine-tuning AraGPT-2 for Egyptian Arabic Dialect → Modern Standard Arabic

Abdelrahman Ahmed · Adham Ashraf · Ahmed Fekry
AIU University · Course AIE241 — NLP · Supervisor: Ashraf Elsayed

AraGPT-2 | Egyptian Arabic | Dialect NLP | Fine-tuning | MSA Generation

Stanford CS224N Framework Extension

THE PROBLEM

Why Egyptian Arabic Is Hard for AI

Arabic Is NOT One Language

100M+ Egyptians
speak Egyptian Arabic daily — not MSA

MSA (Modern Standard Arabic) is used in newspapers, books & NLP models.

The GAP is massive:

Phonology	mish ≠ laysa ("is not")
Loanwords	okay, merci, bravo
Interrogatives	eih ≠ maza ("what")
Tone particles	khalas, yalla, tab — no MSA match

What This Means for NLP

Translation fails

Models produce irrelevant MSA or web-crawl text

Chatbots break

Assistants can't understand or respond to Egyptian input

NLP benchmarks lie

Our Solution: Fine-tune AraGPT-2 to bridge the Egyptian ↔ MSA gap

DATASET

The Data: 18,250 Parallel Sentence Pairs

18,250
Parallel pairs
(Egyptian ↔ MSA)

36,500
Detection examples
(with hard negatives)

80 / 10 / 10
Train / Dev / Test
split ratio

14,600
Training pairs
for fine-tuning

How the Dataset Was Built

1. Original Egyptian→English corpus (Abdalrahmankamel)
2. Keep Egyptian Arabic sentences, drop English translations
3. Add new MSA translations manually
4. Normalize: remove diacritics (tashkeel), clean punctuation

Hard Negative Sampling (Detection Only)

For each Egyptian sentence, a **mismatched MSA sentence from a different example** is paired as a negative example.
This forces the classifier to learn fine-grained semantic matching — not just surface-level patterns.
Result: 50% correct pairs + 50% plausible-but-wrong pairs = balanced binary classification dataset.

ARCHITECTURE

AraGPT-2: The Backbone

Both tasks are built on AraGPT-2 — an Arabic-native GPT-2 transformer pretrained on 77 GB of Arabic web text.

Input Tokens
Egyptian Arabic text
tokenized into subwords

Token Embeddings
Each token → dense vector
(learned during pretraining)

Positional Encoding
Adds position info since
Transformers have no order

12 × Transformer Block
Self-attention + FFN
Captures long-range context

Output Hidden States
One vector per token
dim d = 768 (medium) / 512 (base)

Two Variants Used

aragpt2-medium

Used for: Generation task
~355M parameters
12 layers · 768 hidden dim · 12 heads
Max context: 1,024 tokens

aragpt2-base

Used for: Detection task
~135M parameters
12 layers · 512 hidden dim · 8 heads
Max context: 1,024 tokens

Both are decoder-only (GPT-style): predict next token left → right

TASK 1 — GENERATION

Dialect-to-MSA Generation Pipeline

1

Format Prompt

[Egyptian]: {x}
[MSA]: {y} <eos>

2

Mask Prompt Loss

Set prompt token labels to −100 (ignored by gradient)

3

AraGPT-2 Medium

Autoregressive LM predicts MSA tokens one by one

4

Nucleus Sampling

T=0.7, top-k=50
top-p=0.92
rep. penalty=1.3

The Loss Function

$$L_{gen} = -\sum \log p(y_t \mid [Egyptian]:x, y<t)$$

Only MSA output tokens **(y_t)** contribute to the gradient.
The Egyptian prompt is masked with label=−100 so the model learns:

"Given this Egyptian input, predict the correct MSA word-by-word."

Inference Decoding Params

Temperature T=0.7:	Flattens prob. dist. slightly — more diverse outputs
top-k = 50:	Only sample from top 50 tokens
top-p = 0.92:	Keep tokens covering 92% of prob. mass
Rep. penalty 1.3:	Penalise repeating the same words

TASK 2 — DETECTION

Dialect Translation Detection Pipeline

Cloze Prompt

[Egyptian]: "slang text"
[MSA]: "formal candidate"
Is this correct? [Yes/No]:

AraGPT-2 Base Encoder

Full prompt encoded
(max 160 tokens)
Decoder-only transformer

Extract h_last ∈ $\mathbb{R}^d$

Hidden state of the
FINAL non-padding token
Captures the whole prompt

Linear Classifier Head

z = W·h_last + b
W ∈ $\mathbb{R}^{2 \times d}$, b ∈ $\mathbb{R}^2$
Softmax → [P(yes), P(no)]

✓ CORRECT translation

✗ INCORRECT translation

Why Extract the LAST Token?

GPT-2 is a causal (left-to-right) model.
The last token has attended to ALL previous tokens.
So its hidden state is a compressed summary of the entire prompt — including both the Egyptian input and the MSA candidate.
This single vector is all the classifier needs.

Binary Cross-Entropy Loss

$$L_{det} = -[\, y \cdot \log P(1|x,s) + (1-y) \cdot \log P(0|x,s) \,]$$

y=1 for correct translations, y=0 for hard negatives.
The classifier head W is randomly initialized and trained from scratch; the AraGPT-2 weights are fine-tuned jointly.

TRAINING SETUP

Hyperparameters & Their Meaning

Hyperparameter	Generation	Detection	What It Means
Base model	**aragpt2-medium**	**aragpt2-base**	Larger model for harder generative task
Learning rate	**5×10^{-5}**	**2×10^{-5}**	How fast weights update — lower = safer on small data
Batch size	**8 (eff. 32)**	**16**	Samples per gradient step; gradient accum. ×4 for gen.
LR schedule	**Cosine**	**Linear**	Cosine: smooth decay to ~0. Linear: steady drop.
Warmup ratio	**10%**	**10%**	Ramp up LR first 10% of steps — prevents instability
Weight decay	**0.01**	**0.01**	L2 regularization — reduces overfitting
Max epochs	**10 (stopped at 5)**	**8 (ran full)**	Early stopping on dev loss (patience = 3)
Max input len	**64 tokens**	**160 tokens**	Detection needs longer context for both sentences
Grad. clipping	**max norm 1.0**	**none**	Prevents exploding gradients on small corpus

EVALUATION METRICS

What the Metrics Actually Mean

chrF

Character n-gram F-score

WHAT

Measures character-level overlap between generated MSA and the reference translation.

HOW

Counts shared character sequences (not whole words). More forgiving of morphological variation in Arabic.

OUR RESULT

0–100. Higher = better.
Ours: 29.08 vs baseline 10.62.
+18.5 points improvement.

BLEU

Bilingual Evaluation Understudy

WHAT

Measures n-gram precision: how many word-chunks in the output appear in the reference.

HOW

Counts 1-gram through 4-gram matches, applies brevity penalty for too-short outputs.

OUR RESULT

0–100. Higher = better.
Ours: 6.63 vs baseline 0.02.
Low but normal for dialect→MSA.

PPL

Perplexity

WHAT

How surprised the model is by the test data. Lower = model predicts tokens more confidently.

HOW

PPL = exp(average negative log-likelihood per token) over the FULL sequence.

OUR RESULT

Lower = better (usually).
Ours INCREASED: 11.8M → 59.1M.
See next slide for why.

Detection uses Accuracy: correct predictions / total examples. Random baseline = 0.500. Fine-tuned = 0.956.

PERPLEXITY ANOMALY

Why Did Perplexity INCREASE After Fine-Tuning?

Zero-shot PPL

11,851,030

→

Fine-tuned PPL

59,104,078

This looks bad.
But it is NOT.
Here is why:

1 What PPL actually measures

Perplexity = how well the model predicts EVERY token in the test sequence.
The test sequence includes BOTH the Egyptian prompt AND the MSA output.

2 What fine-tuning does to the model

Fine-tuning shifts the model's distribution toward MSA. The Egyptian prompt tokens (which are masked from the loss with label=−100) NEVER receive gradient updates.
So the model becomes WORSE at predicting Egyptian tokens.

3 Why this raises perplexity

When evaluating, PPL is computed over the FULL sequence.
The model now finds the Egyptian prompt surprising (low probability) → PPL spikes.
This would happen even with 10x more data — it is a measurement artifact.

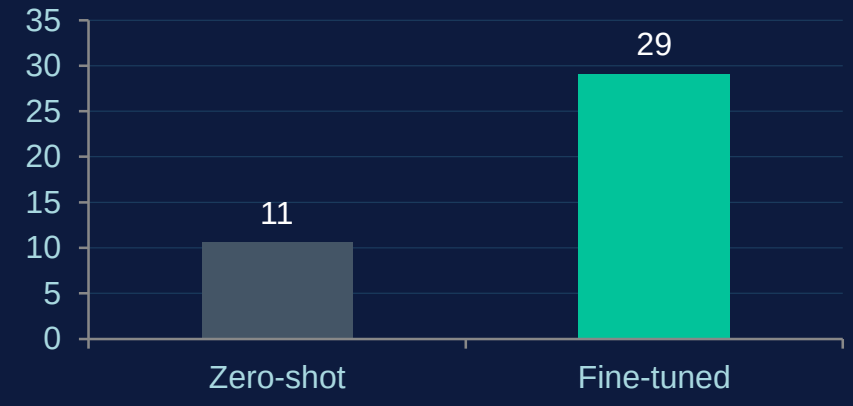
4 The proof it is not a real problem

chrF went from 10.62 → 29.08 (+173%).
BLEU went from 0.02 → 6.63 (+33,000%).
Translation quality clearly improved. PPL is just the wrong metric for conditional fine-tuned models.

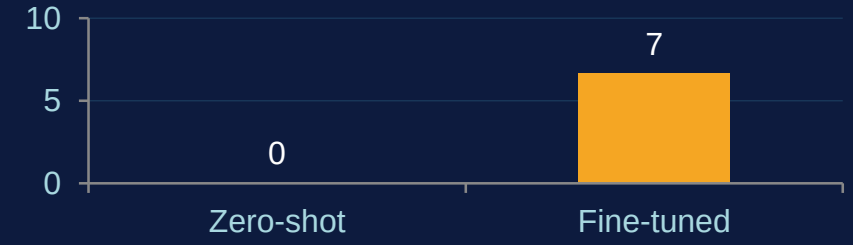
RESULTS

Results: Before vs After Fine-Tuning

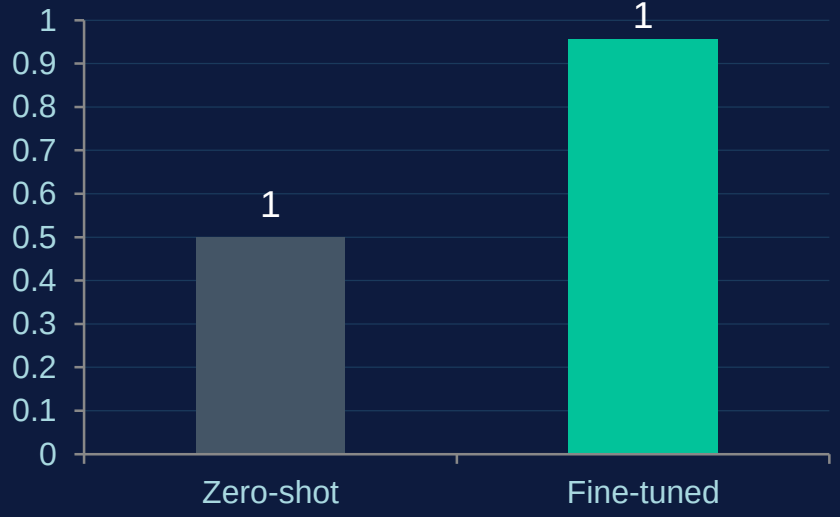
Generation (chrF)



BLEU Score



Detection (Accuracy)



vs Stanford (Hernandez & Naik)

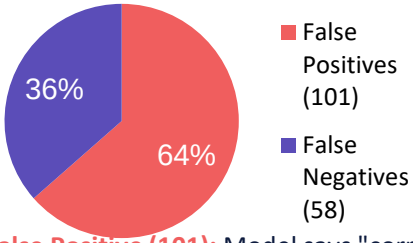
Detection: 0.956 ours vs 0.965 theirs (nearly identical!)

Generation chrF: 29.08 ours vs 41.9 theirs *(lower — cross-register is harder than same-author style)*

ANALYSIS

Error Analysis & Qualitative Results

Detection: 159 Errors on 3,650 Test Examples



False Positive (101): Model says "correct" for a wrong translation. Happens with ambiguous 1-word inputs like "hadir", "okay"

False Negative (58): Model says "wrong" for a correct translation.

Generation Overfitting

Train loss: 2.5 → 0.76 over 5 epochs
Dev loss: min at epoch 3 (2.39), then rises
→ Best checkpoint = epoch 3
Small corpus (14,600 pairs) limits generalization.

Generation Qualitative Examples

Egyptian: yalla feen? (come on, where?)

Zero-shot: ...what good has this blessed month brought... [FORUM DRIFT]

Fine-tuned: hayya, ayna anta? (Come, where are you?)

Egyptian: ana mehtag atkallim ma'aki (I need to talk to you)

Zero-shot: ...oh my soul, you are mine... [SOCIAL MEDIA DRIFT]

Fine-tuned: ahtaj an atahaddatha ma'aki (I need to speak with you)

Egyptian: kunt fakrak mish gayya (I thought you weren't coming)

Zero-shot: ...she who was never betrothed... [FORUM DRIFT]

Fine-tuned: kunnu adhkuruki, lasti qadima [close but misses 'zan']

CONCLUSION

Key Takeaways

01 Fine-tuning works

Both tasks improved massively. Detection: +45.6 pp. Generation chrF: +18.5 pts. Zero-shot AraGPT-2 is nearly useless for these tasks.

02 Cloze-style prompting transfers

The detection formulation from English slang (Hernandez & Naik) transfers almost perfectly to Arabic dialect — 0.956 vs 0.965.

03 Perplexity is NOT translation quality

PPL increased because Egyptian prompt tokens are never trained on — it is a measurement artifact, not a quality failure.

04 Small data limits generation

The 14,600-pair corpus causes overfitting at epoch 3. Larger corpora or encoder-decoder models (AraT5) are the next step.